

الكتاب الفردي (مثال) . المعلوماتية

## Git وإدارة الإصدارات والذكاء الاصطناعي

مثال طالب

مثال مجهول الهوية . الكتاب بلغة الطالب الأم

# النسخة العربية — Git وإدارة الإصدارات والذكاء الاصطناعي

## 1. الأدوات الثلاث وكيف ترتبط ببعضها

1. **Git و GitHub**: المستودع المرتب للعمل، حيث يُحفظ كل إصدار ويمكن مشاركته.
  2. **Gemini**: ذكاء اصطناعي توليدي تتحدث إليه؛ تعطيه نصاً (أو ملف `.md`) فيشرح لك ويترجم ويُلخص.
  3. **NotebookLM**: مُعلّم يجيب فقط اعتماداً على المستندات التي ترفعها (مثل ملف PDF)، لذا يبقى وفيّاً للمصدر.
- الفكرة الأساسية: **Git** يحفظ، و**Gemini** يشرح، و**NotebookLM** يراجع على المصدر. هي ليست أدوات متنافسة بل تُستعمل معاً.

القاعدة الذهبية للدورة: الذكاء الاصطناعي يساعدك على الفهم بشكل أفضل، لا على النسخ أو تخطّي التفكير. الاختبار الحاسم: إذا استطعت أن تشرح بكلماتك ما فعلته ولماذا، فقد فهمت حقاً.

## 2. Git و GitHub

1. **Git** نظام لإدارة الإصدارات: يحفظ تاريخ العمل، فيبقى كل حفظ مهم ويمكن الرجوع إلى الوراء.
  2. **GitHub** هو الموقع على الإنترنت حيث يعيش العمل ويُشارك: أي «Git على الإنترنت». انتبه: **Git** هو الأداة (المحرك)، و**GitHub** هو الخدمة على الإنترنت (المربّاب على الشبكة).
  3. المستودع (**repository**): مجلّد المشروع وبداخله كل الملفات وتاريخها كاملاً.
  4. الالتزام (**commit**): حفظ بعلامة، يلتقط «صورة» للمشروع في لحظة معيّنة مع رسالة تصف ما تغيّر. وتُكوّن الالتزامات المتتابعة تاريخ المشروع.
  5. الفرع (**branch**): مسار عمل موازٍ للتجربة دون إفساد المسار الرئيسي (**main**)؛ ودمج الفرع الرئيسي يُسمّى **merge**.
  6. طلب الدمج (**Pull Request**): اقتراح دمج فرعك في المسار الرئيسي بعد أن يراجع أحدهم.
- نوع الخدمة: **PaaS** (منصة تخدم) — تقدّم لك أحدهم منصّة جاهزة على الإنترنت فتستعملها دون بناء الخوادم وصيانتها. ونموذج الدفع **Freemium**: الوظائف الأساسية مجانية والمتقدّمة بمقابل؛ ويكفيك كطالب الجزء المجاني.

## 3. إدارة الإصدارات: الأرقام 1.2.3

1. **major** (الرقم الأول): يزداد عند تغييرات كبيرة قد لا تتوافق مع القديم.

2. minor (الرقم الثاني): يزداد عند إضافة وظيفة جديدة مع بقاء الباقي يعمل.

3. patch (الرقم الثالث): يزداد عند إصلاح خطأ فقط دون جديد.

مثال: v1.0.0 أول نسخة قابلة للعب ← إضافة الأصوات v1.1.0 ← إصلاح خطأ v1.1.1 ← إعادة كل شيء  
v2.0.0.

القاعدة الذهبية: الإصدار المنشور لا يُعاد استعماله أبداً. إن غيّرت شيئاً واحداً، ارفع الرقم أولاً ثم انشر.

## 4. حالات استخدام Git (الرسوم موصوفة بالكلمات)

1. تطوير خطّي: شخص واحد، بلا فروع، الالتزامات في صفّ والتاريخ خطّ مستقيم.
2. وظيفة على فرع ثم دمج: تطوّر الوظيفة على فرع منفصل والرئيسي يبقى سليماً؛ وعند الجهوزية تدمجها.
3. وظيفتان بالتوازي: فرعان يبدآن معاً وينتهيان في وقتين؛ والرئيسي يجمع القطع تبعاً.
4. إصلاح عاجل (hotfix): أثناء وظيفة طويلة يظهر خطأ خطير، تفتح فرعاً قصيراً وتصلحه فوراً ثم تعود.
5. الإصدارات المنشورة: تنشر إصدارات على الرئيسي (v1.1، v1.0... حتى تغيير كبير v2.0)، وكلّ إصدار «مجمّد».
6. فرع مهجور: تجرب فكرة ولا تدمجها أبداً، والرئيسي لا يتأثر إطلاقاً.
7. فرع من فرع آخر: وظيفة كبيرة تُخرج فرعاً فرعياً؛ يُدمج الفرعي في الفرع ثم الفرع في الرئيسي.
8. تكامل كبير (مشروع جماعي): لكل فرد فرع (رسوم، حركة، صوت، قائمة) ثم تُدمج كلّها في الرئيسي عبر طلبات دمج فتنشأ النسخة الكاملة.
9. فرع صيانة طويل الأمد: يتّجه الرئيسي نحو v2.0، وتبقى فرعاً موازياً بإصلاحات فقط لمن ما زال على v1.
10. revert (إلغاء تعديل خاطئ): لا تُحذف التاريخ، بل تضيف التزاماً جديداً يلغي الخاطئ مع بقاء الأثر بصدق.
11. تعارض دمج وحلّه: فرعان غيرا السطر نفسه، فيتوقّف Git ويقرّر شخص ما ماذا يُبقي وينهي الدمج.
12. فريق بمخسة فروع وإصدارين: الرئيسي ينشر الإصدارات وأربعة فروع تعمل بالتوازي وتعود في أوقات مختلفة.
13. فروع متداخلة بمخسة مستويات: يُقسّم العمل المعقد طبقة داخل طبقة، وكلّ مستوى يعود إلى سابقه كاللّدى المتداخلة.
14. صيانة إصدارين رئيسيين معاً: فرع صيانة لـ v1.x وآخر لـ v2.x موجودان في آن واحد.
15. تاريخ واقعي: نشر، ووظيفة طويلة بفرع فرعي، وإصلاح عاجل، وإلغاء، ودمج بتعارض — كلّها معاً.

## 5. ملفات (Markdown (.md)

1. ملف Markdown (الامتداد .md) هو نصّ خالص برموز قليلة للتنسيق: # للعناوين، و \*\* للتخشين، و سطر يبدأ ب 1. لقائمة مرقّة.
2. يُقرأ في كلّ مكان وبلا صور ثقيلة، فيه المحتوى فقط. لذلك يقرأه الذكاء الاصطناعي جيّداً: نقدّم المذكرات أيضاً بصيغة md. لنعطها ل Gemini.

## 6. Gemini: الذكاء التوليدي، وكيف تستعمله مع ملف .md

1. Gemini ذكاء توليدي: تكتب طلباً (prompt) فيولّد جواباً — يشرح ويترجم ويلخص ويضع أسئلة مراجعة.
  2. انتبه: قد يخطئ أو يختلق («هلوسة»)، فتحقق ممّا يقوله، خاصّة التواريخ والأرقام والمصادر.
  3. الخطوات: افتح [gemini.google.com](https://gemini.google.com) بحساب المدرسة؛ افتح ملف .md وحدّد الكلّ وانسخه؛ ألصقه في Gemini واكتب طلبك.
- أيّ نسخة تختار: Flash (سريعة للاستعمال اليومي)، Pro (أقوى للنصوص المعقّدة)، نسخة الاستدلال (تُفكّر قبل الإجابة، للمسائل الصعبة). القاعدة العملية: ابدأ ب Flash، وإن كان الجواب معقّداً أو خاطئاً فانتقل إلى Pro أو الاستدلال. انظر إلى وظيفتها (سريعة / قوية / تُفكّر) لا إلى الرقم فقط.

## 7. NotebookLM: مُعلّم على المصادر، وكيف تستعمله مع PDF

1. يجب NotebookLM اعتماداً على ما ترفعه فقط: ترفع PDF فيجيبك من ذلك الملف ولا يختلق من خارجه.
2. لذلك هو ممتاز للدراسة والمراجعة على مصدر محدّد، ويبقى وفيّاً للنصّ.
3. الخطوات: افتح [notebooklm.google.com](https://notebooklm.google.com) بحساب المدرسة؛ اضغط «إنشاء جديد» ثم «رفع» واختر PDF؛ اطرح سؤالاً في المحادثة، مثل: «أعطني ملخصاً بسيطاً لهذا المستند».

## 8. المسار الكامل: الأدوات الثلاث معاً

1. تُوضع مواد الدورة على **Git/GitHub** (مُدارة بالإصدارات).
2. تُنزّل من هناك ملف PDF ( للقراءة/الطباعة) أو ملف .md (للذكاء الاصطناعي).
3. يُعطى .md إلى Gemini ليشرح أو يترجم.
4. يُرفع PDF إلى NotebookLM للسؤال والمراجعة على المصدر.

5. يُسَلَّم العمل المنجَز على Classroom.

## 9. الاستعمال المسؤول للذكاء الاصطناعي

1. الفهم لا النسخ: الذكاء الاصطناعي عون على التعلّم لا وسيلة لتخطّي العمل. الاختبار: أن تشرح بكلماتك.
2. التحقق دائماً: قد يخطئ أو يختلق، فتحقّق ممّا يقول.
3. الخصوصية والبيانات: لا تُدخِل بيانات شخصية أو خاصّة بالزملاء أو العائلة في هذه المساعدات.

مثال تقييم: الدرجة 90/100 – العمل كامل وصحيح، وفهم واضح لـ Git وإدارة الإصدارات، مع القدرة على الشرح بالكلمات الخاصة. (مثال مجهول الهوية)